

Điều này chỉ ra rằng phép nhân hai số nguyên $2n$ bit có thể thực hiện bằng cách dùng 3 phép nhân các số nguyên n bit và các phép cộng, trừ dịch chuyển. Như vậy, nếu $f(n)$ là tổng các phép toán nhị phân cần thiết để nhân hai số n bit thì

$$f(2n) = 3f(n) + Cn$$

Ba phép nhân các số nhị phân n bit cần $3f(n)$ phép toán nhị phân. Mỗi một phép toán cộng, trừ, dịch chuyển dùng một hằng số nhân với n lần chính là Cn .

Ví dụ 2. Bài toán xếp khách của Lucas. Có một bàn tròn, xung quanh có $2n$ ghế. Cần sắp chỗ cho n cặp vợ chồng sao cho các ông ngồi sen kẽ các bà và không có hai cặp vợ chồng nào ngồi cạnh nhau. Hỏi có tất cả bao nhiêu cách xếp?

Lời giải. Gọi số phải tìm là M_n . Xếp cho các bà trước (cứ xếp một ghế thì một ghế để trống dành cho các ông), số cách xếp cho các bà là $2n!$ cách. Gọi số cách xếp cho các ông ứng với một cách xếp các bà là U_n ta được số cách xếp là

$$M_n = 2n! \times U_n. \text{ Vấn đề còn lại là tính số } U_n.$$

Đánh số các bà (đã xếp) từ 1 đến n , đánh số các ông tương ứng với các bà (ông i là chồng bà i), sau đó đánh số các ghế trống theo nguyên tắc: ghế số i nằm giữa bà i và bà $i+1$ (các phép cộng được hiểu lấy modul n nghĩa là $n+1=1$). Mỗi cách xếp các ông được biểu diễn bằng một phép thế φ trên tập $\{1, 2, \dots, n\}$ với qui ước $\varphi(i) = j$ có nghĩa là ghế i được xếp cho ông j . Theo giả thiết φ phải thoả mãn:

$$\varphi(i) \neq i \text{ và } \varphi(i) \neq i+1 \quad (*)$$

Như vậy, U_n là số tất cả các phép thế φ thoả mãn điều kiện (*). Trong toán học gọi U_n là số phân bố.

Xét tập hợp tất cả các phép thế φ của $\{1, 2, \dots, n\}$. Trên tập này ta gọi P_i là tính chất $\varphi(i) = i$, Q_i là tính chất $\varphi(i) = i+1$. Đặt $P_{n+1} = Q_i$, theo nguyên lý bù trừ tương ứng với $2n$ tính chất P_i ta có:

$U_n = \bar{N} = n! - N_1 + N_2 + \dots$ trong đó N_k là tổng số tất cả các phép thế thoả mãn k tính chất lấy từ $2n$ tính chất đang xét. Cần chú ý rằng, không thể xảy ra đồng thời thoả mãn P_i và Q_i . Do đó trong các phép lấy ra k tính chất từ $2n$ tính chất đang xét cần thêm vào điều kiện: P_i và Q_i hoặc P_{i+1} và Q_i không được đồng thời có mặt. Gọi số các cách này là $g(2n, k)$ (nói riêng $g(2n, k) = 0$ khi $k > n$). Với mỗi cách lấy ra k tính chất như vậy ($k \leq n$) ta có $(n-k)!$ phép thế thoả mãn chúng. Từ đó ta nhận được $N_k = g(2n, k) (n-k)!$ và

$$U_n = n! - g(2n, 1)(n-1)! + g(2n, 2)(n-2)! - \dots + (-1)^n g(2n, n)$$

Bây giờ chúng ta phải tính các hệ số $g(2n, k)$, $k = 1, 2, \dots, n$.

Xếp $2n$ tính chất đang xét trên vòng tròn theo thứ tự $P_1, Q_1, P_2, Q_2, \dots, P_n, Q_n$, ta thấy rằng $g(2n, k)$ chính là số cách lấy k phần tử trong $2n$ phần tử xếp thành vòng tròn sao cho không có hai phần tử nào kề nhau cùng được lấy ra. Để tính $g(2n, k)$ ta giải hai bài toán con sau:

Bài toán 1. Có bao nhiêu cách lấy ra k phần tử trong n phần tử xếp trên đường thẳng sao cho không có hai phần tử nào kề nhau cùng được lấy ra.

Lời Giải. Khi lấy k phần tử, ta còn $n-k$ phần tử. Giữa $n-k$ phần tử còn lại có $n-k+1$ khoảng trống (kể cả hai đầu). Mỗi cách lấy ra k khoảng từ các khoảng này sẽ tương ứng với một cách chọn k phần tử thỏa mãn yêu cầu đã nêu. Vậy số cách chọn cần tìm là $C(n-k+1, k)$.

Bài toán 2. Giống như bài toán 1 nhưng n phần tử xếp trên vòng tròn.

Lời giải. Cố định phần tử a được chọn chia các cách lấy thành 2 lớp

- 1- Các cách mà a được chọn khi đó 2 phần tử kề a sẽ không được chọn và phải lấy $k-1$ phần tử từ $n-3$ phần tử còn lại. Các phần tử này xem như kết quả của bài toán 1. Theo bài toán 1, số cách thuộc lớp kiểu này là $C(n-k-1, k-1)$.
- 2- Các cách mà a không được chọn, khi đó bỏ a đi và bài toán trở về bài toán 1 chọn k phần tử từ $n-1$ phần tử xếp trên đường thẳng. Theo bài toán 1 số cách xếp kiểu này là $C(n-k, k)$.

Vậy theo nguyên lý cộng số cách cần tìm là :

$$C(n-k-1, k-1) + C(n-k, k) = \frac{n}{n-k} C(n-k, k)$$

Từ kết quả của hai bài toán trên ta nhận được:

$$g(2n, k) = \frac{2n}{2n-k} C(2n-k, k) \text{ và số phân bố } U_n \text{ được tính bằng}$$

$$U_n = n! - \frac{2n}{2n-1} C(2n-1, 1)(n-1)! + \frac{2n}{2n-2} C(2n-2, 2)(n-2)! - \dots + (-1)^n \frac{2n}{n} C(2n, n)$$

Dưới đây là một số giá trị của U_n , một lần nữa chúng ta lại được quan sát hiện tượng bùng nổ tổ hợp.

n	2	3	4	5	6	7	8	9	10
U_n	0	1	2	13	80	579	4783	43387	439792

2.6. Phương pháp liệt kê

Việc tìm một công thức cho kết quả đếm ngay cả trong trường hợp công thức truy hồi không phải dễ dàng và lúc nào cũng thực hiện được. Cho đến nay còn nhiều bài toán đếm chưa có lời giải dưới dạng một công thức. Đối với những bài toán như vậy, người ta chỉ còn cách chỉ ra một phương pháp liệt kê, theo đó có thể đi qua được tất cả các cấu hình cần đếm. Rõ ràng bản thân phương pháp liệt kê không chỉ ra được một kết quả cụ thể nào nhưng qua đó người ta có thể lập trình cho máy tính điện tử đếm hộ.

Để minh họa cho phương pháp liệt kê, ta xét một cấu hình tổ hợp nổi tiếng đó là các hình chữ nhật la tinh.

Giả sử S là tập gồm n phần tử. Không mất tính tổng quát ta giả sử $S = \{1, 2, \dots, n\}$. Một hình chữ nhật la tinh trên S là một bảng gồm p dòng, q cột sao cho mỗi dòng của nó là một chỉnh hợp không lặp chập q của S và mỗi cột của nó là một chỉnh hợp không lặp chập p của S .

Theo định nghĩa ta có $p \leq n$, $q \leq n$. Đặc biệt trong trường hợp $q = n$, mỗi dòng của hình chữ nhật la tinh là một hoán vị của S , sao cho không có cột nào chứa hai phần tử lặp lại. Hình chữ nhật la tinh dạng này được gọi là chuẩn nếu dòng đầu của nó là hoán vị $1, 2, \dots, n$.

Ví dụ. Hình la tinh chuẩn trên tập $S = \{1, 2, 3, 4, 5, 6, 7\}$

1	2	3	4	5	6	7
2	3	4	5	6	7	1
3	4	5	6	7	1	2

Gọi $L(p, n)$ là số hình chữ nhật la tinh $p \times n$, còn $K(p, n)$ là số hình chữ nhật la tinh chuẩn $p \times n$ ta có:

$$L(p, n) = n! K(p, n)$$

Dễ dàng nhận thấy rằng, số D_n là số hình la tinh chuẩn $2 \times n$, số phân bố U_n là số hình chữ nhật la tinh chuẩn $3 \times n$ với hai dòng đầu là:

1	2	...	$n-1$	n
2	3	...	n	1

Riordan J(1946) đã chứng minh công thức:

$$K(3, n) = \sum_{k=0}^m C(n, k) D_{n-k} D_k U_{n-2k} \text{ trong đó } m = [n/2], U_0 = 1.$$

Bài toán đếm với số dòng nhiều hơn đến nay vẫn chưa được giải quyết. Người ta mới chỉ đưa ra được một vài dạng tiệm cận của $L(p, n)$.

Nếu $p=q=n$, thì hình chữ nhật la tinh được gọi là hình vuông la tinh. Một hình vuông la tinh cấp n được gọi là chuẩn nếu có dòng đầu và cột đầu là hoán vị $1, 2, \dots, n$. Thí dụ một hình vuông la tinh chuẩn cấp 7.

1	2	3	4	5	6	7
2	3	4	5	6	7	1
3	4	5	6	7	1	2
4	5	6	7	1	2	3
5	6	7	1	2	3	4
6	7	1	2	3	4	5

7 1 2 3 4 5 6

Gọi l_n là số các hình vuông như thế ta có $L(n,n) = n!(n-1)!l_n$

Việc tìm một công thức cho l_n đến nay vẫn bỏ ngỏ. Tuy nhiên ta có thể nhờ máy tính liệt kê tất cả các hình vuông chuẩn cấp n . Dưới đây là một vài giá trị tính được.

N	1	2	3	4	5	6	7
l_n	1	1	1	4	56	9408	16942080

PREL

BÀI TẬP CHƯƠNG 2

1. Xâu thuận nghịch độc là một xâu khi viết theo thứ tự ngược lại cũng bằng chính nó. Hãy đếm số xâu nhị phân có độ dài n là thuận nghịch độc.
2. Cô dâu và chú rể mời bốn bạn đứng thành một hàng để chụp ảnh. Hỏi có bao nhiêu cách xếp hàng nếu:
 - a) Cô dâu đứng cạnh chú rể
 - b) Cô dâu không đứng cạnh chú rể
 - c) Cô dâu đứng ở phía bên phải chú rể
3. Có bao nhiêu xâu nhị phân độ dài 10 có năm số 0 liên nhau hoặc năm số 1 liên nhau.
4. Có bao nhiêu xâu nhị phân độ dài bằng 8 có 3 số 0 liên nhau hoặc 4 số 1 liên nhau.
5. Mỗi sinh viên lớp toán học rời rạc hoặc giỏi toán hoặc giỏi tin học hoặc giỏi cả hai môn này. Trong lớp có bao nhiêu sinh viên nếu 38 người giỏi tin (kể cả người giỏi cả hai môn), 23 người giỏi toán (kể cả người giỏi cả hai môn), và 7 người giỏi cả hai môn.
6. Chứng tỏ rằng, trong $n+1$ số nguyên dương không vượt quá $2n$ tồn tại ít nhất một số chia hết cho một số khác.
7. Chứng minh rằng, trong dãy gồm $n^2 + 1$ số thực phân biệt đều có một dãy con dài $n+1$ hoặc thực sự tăng, hoặc thực sự giảm.
8. Giả sử trong một nhóm 6 người mỗi cặp hai hoặc là bạn, hoặc là thù. Chứng tỏ rằng trong nhóm có ba người là bạn của nhau hoặc là kẻ thù của nhau.
9. Hãy chỉ ra rằng, trong 102 người có chiều cao khác nhau đứng thành một hàng có thể tìm được 11 người có chiều cao tăng dần hoặc giảm dần mà không cần thay đổi thứ tự của họ trong hàng.
10. Một đô vật tay tham gia thi đấu giành chức vô địch trong 75 giờ. Mỗi giờ anh ta thi đấu ít nhất một trận, nhưng toàn bộ anh ta không thi đấu quá 125 trận. Chứng tỏ rằng, có những giờ liên tiếp anh ta thi đấu 24 trận.
11. Một nhân viên bắt đầu làm việc tại công ty từ năm 1987 với mức lương khởi điểm là 50000 đô la. Hàng năm anh ta được nhận thêm 1000 đô la và 5% lương của năm trước.
 - a) Hãy thiết lập hệ thức truy hồi tính lương của nhân viên đó n năm sau năm 1987.
 - b) Lương vào năm 1995 của anh ta là bao nhiêu?
 - c) Hãy tìm công thức tường minh tính lương của nhân viên này n năm sau năm 1987.

12. Tìm hệ thức truy hồi cho số hoán vị của tập n phần tử. Dùng hệ thức truy hồi đó tính hoán vị của tập n phần tử.
13. Một máy bán tem tự động chỉ nhận các đồng xu một đô la và các loại tờ tiền 1 đô la và 5 đô la.
- Hãy tìm hệ thức truy hồi tính số cách đặt n đô la vào trong máy bán hàng, trong đó thứ tự các đồng xu, các tờ tiền là quan trọng.
 - Tìm các điều kiện đầu.
 - Có bao nhiêu cách đặt 10 đô la vào máy để mua một bộ tem.
14. Giải các hệ thức truy hồi với các điều đầu sau:
- $a_n = a_{n-1} + 6a_{n-2}$ với $n \geq 2, a_0 = 3, a_1 = 6$.
 - $a_n = 7a_{n-1} - 6a_{n-2}$ với $n \geq 2, a_0 = 2, a_1 = 1$.
 - $a_n = 6a_{n-1} - 8a_{n-2}$ với $n \geq 2, a_0 = 4, a_1 = 10$.
 - $a_n = 2a_{n-1} - a_{n-2}$ với $n \geq 2, a_0 = 4, a_1 = 1$.
 - $a_n = a_{n-2}$ với $n \geq 2, a_0 = 5, a_1 = -1$.
 - $a_n = -6a_{n-1} - 9a_{n-2}$ với $n \geq 2, a_0 = 3, a_1 = -3$.
 - $a_{n+2} = -4a_{n+1} + 5a_n$ với $n \geq 0, a_0 = 2, a_1 = 8$.
15. Tìm các nghiệm đặc trưng của hệ thức truy hồi tuyến tính thuần nhất
- $a_n = 2a_{n-1} - 2a_{n-2}$
 - Tìm nghiệm thỏa mãn hệ thức truy hồi trên và các điều kiện đầu $a_0 = 1, a_1 = 2$.
16. a) Tìm nghiệm đặc trưng của hệ thức truy hồi tuyến tính thuần nhất $a_n = an - 4$
- Tìm nghiệm thỏa mãn hệ thức truy hồi trên và các điều kiện đầu $a_0 = 1, a_1 = 0, a_2 = -1, a_3 = 1$.
17. Một báo cáo về thị trường máy tính cá nhân cho biết có 65000 người sẽ mua modem cho máy tính của họ trong năm tới, 1 250 000 người sẽ mua ít nhất một sản phẩm phần mềm. Nếu báo cáo này nói rằng 1.450.000 người sẽ mua hoặc là modem hoặc là ít nhất một sản phẩm phần mềm thì sẽ có bao nhiêu người sẽ mua cả modem và mua ít nhất một sản phẩm phần mềm.

CHƯƠNG 3. BÀI TOÁN LIỆT KÊ

Nội dung bài toán đếm là đếm xem có bao nhiêu cấu hình tổ hợp thỏa mãn một số tính chất nào đó. Bài toán liệt kê không chỉ đếm được các cấu hình tổ hợp thỏa mãn các tính chất đặt ra mà còn xem xét từng cấu hình tổ hợp đó là gì. Đối với mỗi bài toán, khi chưa tìm được thuật giải thì liệt kê được xem là biện pháp cuối cùng để thực hiện với sự hỗ trợ của máy tính. Có thể nói, liệt kê được xem là phương pháp giải vạn năng một bài toán bằng máy tính. Nội dung chính của chương này tập chung giải quyết những vấn đề cơ bản sau:

- ✓ Giới thiệu bài toán liệt kê.
- ✓ Thuật toán và độ phức tạp tính toán.
- ✓ Giải quyết bài toán liệt kê bằng phương pháp sinh.
- ✓ Giải quyết bài toán liệt kê bằng phương pháp quay lui.

Bạn đọc có thể tìm thấy cách giải nhiều bài toán liệt kê trong các tài liệu [1] và [2] trong tài liệu tham khảo.

3.1- Giới thiệu bài toán

Bài toán đưa ra danh sách tất cả các cấu hình tổ hợp có thể có được gọi là bài toán liệt kê tổ hợp. Khác với bài toán đếm là tìm kiếm một công thức cho lời giải, bài toán liệt kê lại cần xác định một thuật toán để theo đó có thể xây dựng được lần lượt tất cả các cấu hình cần quan tâm. Một thuật toán liệt kê phải đảm bảo hai nguyên tắc:

- ☐ Không được lặp lại bất kỳ một cấu hình nào.
- ☐ Không được bỏ sót bất kỳ một cấu hình nào.

Ví dụ 1. Cho tập hợp các số $a_1, a_2, \dots, a_n$ và số M . Hãy tìm tất cả các tập con k phần tử của dãy số $\{a_n\}$ sao cho tổng số các phần tử trong tập con đó đúng bằng M .

Lời giải: Như chúng ta đã biết, số các tập con k phần tử của tập gồm n phần tử là $C(n,k)$. Như vậy chúng ta cần phải duyệt trong số $C(n,k)$ tập k phần tử để lấy ra những tập có tổng các phần tử đúng bằng M . Vì không thể xác định được có bao nhiêu tập k phần tử từ tập n phần tử có tổng các phần tử đúng bằng M nên chúng ta chỉ còn cách liệt kê các cấu hình thỏa mãn điều kiện đã cho.

Ví dụ 2. Một thương nhân đi bán hàng tại tám thành phố. Chị ta có thể bắt đầu hành trình của mình tại một thành phố nào đó nhưng phải qua 7 thành phố kia theo bất kỳ thứ tự nào mà chị ta muốn. Hãy chỉ ra lộ trình ngắn nhất mà chị ta có thể đi.

Lời giải: Vì thành phố xuất phát đã được xác định. Do vậy thương nhân có thể chọn tùy ý 7 thành phố còn lại để hành trình. Như vậy, tất cả số hành trình của thương nhân có thể

đi qua là $7! = 5040$ cách. Tuy nhiên trong 5040 cách chúng ta phải duyệt toàn bộ để chỉ ra một hành trình là ngắn nhất.

Có thể nói phương pháp liệt kê là biện pháp cuối cùng nhưng cũng là biện pháp phổ dụng nhất để giải quyết các bài toán tổ hợp. Khó khăn chính của phương pháp này là sự bùng nổ tổ hợp. Để xây dựng chừng 1 tỷ cấu hình (con số này không phải là lớn đối với các bài toán tổ hợp như số mất thứ tự D_n , số phân bố U_n , số hình vuông la tinh ln), ta giả sử cần 1 giây để liệt kê một cấu hình thì chúng ta cũng cần 31 năm mới giải quyết xong. Tuy nhiên với sự phát triển nhanh chóng của máy tính, bằng phương pháp liệt kê, nhiều bài toán khó của lý thuyết tổ hợp đã được giải quyết và góp phần thúc đẩy sự phát triển của nhiều ngành toán học.

3.2. Thuật toán và độ phức tạp tính toán

3.2.1. Ví dụ và Định nghĩa

Định nghĩa. Dãy hữu hạn các thao tác sơ cấp $F=F_1F_2..F_n(\text{Input}) \rightarrow \text{Output}$ được gọi là một thuật toán trên tập thông tin vào Input để có được kết quả ra Output. Dãy các thao tác sơ cấp ở đây được hiểu là các phép toán số học, các phép toán logic, các phép toán so sánh.

Một thuật toán cần thỏa mãn các tính chất dưới đây:

- **Tính đơn định.** Ở mỗi bước của thuật toán, các thao tác sơ cấp phải hết sức rõ ràng, không gây nên sự lộn xộn, nhập nhằng, đa nghĩa. Thực hiện đúng các bước của thuật toán trên tập dữ liệu vào, chỉ cho duy nhất một kết quả ra.
- **Tính dừng.** Thuật toán không được rơi vào quá trình vô hạn. Phải dừng lại và cho kết quả sau một số hữu hạn các bước.
- **Tính đúng.** Sau khi thực hiện tất cả các bước của thuật toán theo đúng qui trình đã định, ta phải nhận được kết quả mong muốn với mọi bộ dữ liệu đầu vào. Kết quả đó được kiểm chứng bằng yêu cầu của bài toán.
- **Tính phổ dụng.** Thuật toán phải dễ sửa đổi để thích ứng được với bất kỳ bài toán nào trong lớp các bài toán cùng loại và có thể làm việc trên nhiều loại dữ liệu khác nhau.
- **Tính khả thi.** Thuật toán phải dễ hiểu, dễ cài đặt, thực hiện được trên máy tính với thời gian cho phép.

3.2.2. Phương pháp biểu diễn thuật toán:

Thông thường, để biểu diễn một thuật toán ta có thể sử dụng các phương pháp sau:

- **Biểu diễn bằng ngôn ngữ tự nhiên.** Ngôn ngữ tự nhiên là phương tiện giao tiếp giữa con người với con người. Ta có thể sử dụng chính ngôn ngữ này vào việc biểu diễn thuật toán.

- **Ngôn ngữ hình thức.** Ngôn ngữ hình thức là phương tiện giao tiếp trung gian giữa con người và hệ thống máy tính. Ví dụ ngôn ngữ sơ đồ khối, ngôn ngữ tựa tự nhiên, ngôn ngữ đặc tả. Đặc điểm chung của các loại ngôn ngữ này là việc sử dụng nó rất gần với ngôn ngữ tự nhiên và ngôn ngữ máy tính.
- **Ngôn ngữ máy tính.** Là phương tiện giao tiếp giữa máy tính và máy tính. Trong trường hợp này ta có thể sử dụng bất kỳ ngôn ngữ lập trình nào để mô tả thuật toán.

Ghi chú. Trong các phương pháp biểu diễn thuật toán, phương pháp biểu diễn bằng ngôn ngữ hình thức được sử dụng rộng rãi vì nó gần với ngôn ngữ tự nhiên và không phụ thuộc vào ngôn ngữ máy tính.

Ví dụ 1. Biểu diễn thuật toán tìm USCLN (a, b) bằng ngôn ngữ tự nhiên.

Đầu vào (Input). Hai số tự nhiên a, b.

Đầu ra (Output). Số nguyên u lớn nhất để a và b đều chia hết cho u.

Thuật toán (Euclidean Algorithm):

Bước 1. Đưa vào hai số tự nhiên a và b.

Bước 2. Nếu $b \neq 0$ thì chuyển đến bước 3, nếu $b=0$ thì thực hiện bước 4.

Bước 3. Đặt $r = a \bmod b$; $a = b$; $b = r$; Sau đó quay trở lại bước 2.

Bước 4 (Output). Kết luận $u=a$ là số nguyên cần tìm.

Ví dụ 2. Biểu diễn thuật toán tìm USCLN (a, b) bằng ngôn ngữ hình thức.

Thuật toán Euclide:

Đầu vào (Input): $a \in \mathbb{N}, b \in \mathbb{N}$.

Đầu ra (Output): $s = \max \{ u \in \mathbb{N} : a \bmod u = 0 \text{ and } b \bmod u = 0 \}$.

Format : $s = \text{Euclide}(a, b)$.

Actions :

```
while (b  $\neq$  0 ) do
    r = a mod b; a = b; b = r;
endwhile;
return(a);
```

Endactions.

Ví dụ 3. Biểu diễn thuật toán tìm USCLN (a, b) bằng ngôn ngữ máy tính (C++).

```
Int    USCLN( int a, int b) {
    while ( b  $\neq$  0 ) {
        r = a % b; a = b; b = r;
```

```

    }
    return(a);
}

```

3.2.3. Độ phức tạp tính toán

Một bài toán có thể thực hiện bằng nhiều thuật toán khác nhau. Chọn giải thuật nhanh nhất giải bài toán là một nhu cầu của thực tế. Vì vậy ta cần phải có sự ước lượng cụ thể để minh chứng bằng toán học mức độ nhanh chậm của mỗi giải thuật.

Khái niệm độ phức tạp thuật toán:

Thời gian thực hiện một giải thuật bằng chương trình máy tính phụ thuộc vào các yếu tố:

- Kích thước dữ liệu vào: Dữ liệu càng lớn thì thời gian xử lý càng chậm.
- Phần cứng máy tính: máy có tốc độ cao thực hiện nhanh hơn trên máy có tốc độ thấp. Tuy vậy, yếu tố này không ảnh hưởng đến quá trình xác định thời gian thực hiện của thuật toán nếu xem xét thời gian thực hiện thuật toán như một hàm của độ dài dữ liệu $T(n)$.

Tổng quát, cho hai hàm $f(x)$, $g(x)$ xác định trên tập các số nguyên dương hoặc tập các số thực vào tập các số thực. Hàm $f(x)$ được gọi là $O(g(x))$ nếu tồn tại một hằng số $C > 0$ và n_0 sao cho:

$$|f(x)| \leq C \cdot |g(x)| \text{ với mọi } x \geq n_0.$$

Điều này có nghĩa với các giá trị $x \geq n_0$ hàm $f(x)$ bị chặn trên bởi hằng số C nhân với $g(x)$. Nếu $f(x)$ là thời gian thực hiện của một thuật toán thì ta nói giải thuật đó có cấp $g(x)$ hay độ phức tạp thuật toán là $O(g(x))$.

Ghi chú. Các hằng số C , n_0 thỏa mãn điều kiện trên là không duy nhất. Nếu có đồng thời $f(x)$ là $O(g(x))$ và $h(x)$ thỏa mãn $g(x) < h(x)$ với $x > n_0$ thì ta cũng có $f(x)$ là $O(h(x))$.

Ví dụ 1. Cho $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$. Trong đó, a_i là các số thực ($i = 0, 1, 2, \dots, n$). Khi đó $f(x) = O(x^n)$.

Chứng minh. Thực vậy, với mọi $x > 1$:

$$\begin{aligned}
|f(x)| &= |a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0| \\
&\dots \leq |a_n| x^n + |a_{n-1}| x^{n-1} + \dots + |a_1| x + |a_0| \\
&\dots \leq |a_n| x^n + |a_{n-1}| x^n + \dots + |a_1| x^n + |a_0| x^n \\
&\dots \leq x^n (|a_n| + |a_{n-1}| + \dots + |a_1| + |a_0|) \\
&\dots \leq C x^n = O(x^n). \\
C &= (|a_n| + |a_{n-1}| + \dots + |a_1| + |a_0|)
\end{aligned}$$

PTE

Ví dụ 2. Tìm độ phức tạp thuật toán sắp xếp kiểu Bubble-Sort?

```
Void Bubble-Sort ( int A[], int n ) {  
    for ( i=1; i<n; i++) {  
        for ( j = i+1; j<=n; j++){  
            if (A[i] > A[j]) {  
                t = A[i]; A[i] = A[j]; A[j] = t;  
            }  
        }  
    }  
}
```

Lời giải. Sử dụng trực tiếp nguyên lý cộng ta có:

- Với $i=1$ ta cần sử dụng $n-1$ phép so sánh $A[i]$ với $A[j]$;
- Với $i=2$ ta cần sử dụng $n-2$ phép so sánh $A[i]$ với $A[j]$;
-
- Với $i=n-1$ ta cần sử dụng 1 phép so sánh $A[i]$ với $A[j]$;

Vì vậy tổng số các phép toán cần thực hiện là:

$$S = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 \leq n^2 = O(n^2).$$

Ghi chú. Độ phức tạp thuật toán cũng là số lần thực hiện phép toán tích cực. Phép toán tích cực là phép toán thực hiện nhiều nhất đối với thuật toán.

Một số tính chất của độ phức tạp thuật toán:

- Với $P(n)$ là một đa thức bậc k thì $O(P(n)) = O(n^k)$. Vì thế ta nói, một thuật toán có độ phức tạp cấp đa thức là $O(n^k)$.
- Với a, b là hai cơ số tùy ý và $f(n)$ là một hàm xác định dương thì $\log_a f(n) = \log_a b \cdot \log_b(f(n))$. Vì vậy độ phức tạp thuật toán cấp logarit được ký hiệu là $O(\log(f(n)))$ mà không cần quan tâm đến cơ số.
- Nếu độ phức tạp thuật toán là hằng số, nghĩa là thời gian tính toán không phụ thuộc vào độ dài dữ liệu được ký hiệu là $O(1)$.
- Một giải thuật có cấp $2^n, n!, n^n$ được gọi là giải thuật hàm mũ. Những giải thuật này thường có tốc độ rất chậm.
- Độ phức tạp tính toán của một đoạn chương trình P chính bằng số lần thực hiện một phép toán tích cực. Trong đó, phép toán tích cực trong một đoạn chương trình là phép toán mà số lần thực hiện nó không ít hơn các phép toán khác.

Các dạng hàm đánh giá độ phức tạp thuật toán:

Dạng đánh giá	Tên gọi
$O(1)$	Hằng số
$O(\lg \lg n)$	Log log
$O(\lg n)$	Logarithm
$O(n)$	Tuyến tính
$O(n^2)$	Bậc hai
$O(n^3)$	Bậc 3
$O(nm)$	Đa thức
$O(mn)$	Hàm mũ
$O(n!)$	Giai thừa

3.2.4. Quy tắc xác định độ phức tạp thuật toán

Quy tắc tổng: Nếu $f_1(x)$ có độ phức tạp là $O(g_1(x))$ và $f_2(x)$ có độ phức tạp là $O(g_2(x))$ thì độ phức tạp của $(f_1(x) + f_2(x))$ là $O(\max(g_1(x), g_2(x)))$.

Chứng minh.

- Vì $f_1(x)$ có độ phức tạp là $O(g_1(x))$ nên tồn tại hằng số C_1 và k_1 sao cho $|f_1(x)| \leq |g_1(x)|$ với mọi $x \leq k_1$;
- Vì $f_2(x)$ có độ phức tạp là $O(g_2(x))$ nên tồn tại hằng số C_2 và k_2 sao cho $|f_2(x)| \leq |g_2(x)|$ với mọi $x \leq k_2$;
- Ta lại có :

$$\begin{aligned} |f_1(x) + f_2(x)| &\leq |f_1(x)| + |f_2(x)| \\ &\leq C_1|g_1(x)| + C_2|g_2(x)| \\ &\leq C|g(x)| \text{ với mọi } x > k; \end{aligned}$$

Trong đó, $C = C_1 + C_2$; $g(x) = \max(g_1(x), g_2(x))$; $k = \max(k_1, k_2)$.

Tổng quát. Nếu độ phức tạp của $f_1(x), f_2(x), \dots, f_m(x)$ lần lượt là $O(g_1(x)), O(g_2(x)), \dots, O(g_m(x))$ thì độ phức tạp của $f_1(x) + f_2(x) + \dots + f_m(x)$ là $O(\max(g_1(x), g_2(x), \dots, g_m(x)))$.

Qui tắc nhân: Nếu $f(x)$ có độ phức tạp là $O(g(x))$ thì độ phức tạp của $f^n(x)$ là $O(g^n(x))$. Trong đó:

$$f^n(x) = f(x).f(x)....f(x). //n \text{ lần } f(x).$$

$$g^n(x) = g(x).g(x)...g(x). //n \text{ lần } g(x)$$

Nói cách khác, đoạn chương trình P có thời gian thực hiện $T(n) = O(f(n))$. Khi đó, nếu thực hiện $k(n)$ lần đoạn chương trình P với $k(n)$ là $O(g(n))$ thì độ phức tạp tính toán là $O(f(n).g(n))$.

Chứng minh. Thật vậy theo giả thiết $f(x)$ là $O(g(x))$ nên tồn tại hằng số C và k sao cho với mọi $x > k$ thì $|f(x)| \leq C.g(x)$. Ta có:

$$\begin{aligned} |f^n(x)| &= |f^1(x).f^2(x)...f^n(x)| \\ &\leq |C.g^1(x).C.g^2(x)...C.g^n(x)| \\ &\leq C^n |g^n(x)| = O(g^n(x)) \end{aligned}$$

3.2.5. Độ phức tạp của các cấu trúc lệnh

Để đánh giá độ phức tạp của một thuật toán đã được mã hóa thành chương trình máy tính ta thực hiện theo một số qui tắc sau.

Độ phức tạp hằng số $O(1)$: đoạn chương trình không chứa vòng lặp hoặc lời gọi đệ qui có tham biến là một hằng số.

Ví dụ 1.7. Đoạn chương trình dưới đây có độ phức tạp hằng số.

```
for (i=1; i<=c; i++) {
    <Tập các chỉ thị có độ phức tạp O(1)>;
}
```

Độ phức tạp $O(n)$: Độ phức tạp của hàm hoặc đoạn code là $O(n)$ nếu biến trong vòng lặp tăng hoặc giảm bởi một hằng số c .

Ví dụ 1.8. Đoạn code dưới đây có độ phức tạp hằng số.

```
for (i=1; i<=n; i = i + c ) {
    <Tập các chỉ thị có độ phức tạp O(1)>;
}
for (i=n; i>0; i = i - c ){
    <Tập các chỉ thị có độ phức tạp O(1)>;
}
```

Độ phức tạp đa thức $O(n^c)$: Độ phức tạp của c vòng lặp lồng nhau, mỗi vòng lặp đều có độ phức tạp $O(n)$ là $O(n^c)$.

Ví dụ 1.9. Đoạn code dưới đây có độ phức tạp $O(n^2)$.

```
for (i=1; i<=n; i = i + c ) {
```

```

    for (j=1; j<=n; j = j + c ){
        <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
    }
}
for (i = n; i >0 ; i = i - c ) {
    for (j = i- 1; j>1; j = j -c ){
        <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
    }
}

```

Độ phức tạp logarit $O(\text{Log}(n))$: Độ phức tạp của vòng lặp là $\log(n)$ nếu biểu thức khởi đầu lại của vòng lặp được chia hoặc nhân với một hằng số c .

Ví dụ 1.10. Đoạn code dưới đây có độ phức tạp $\text{Log}(n)$.

```

for (i=1; i <=n; i = i *c ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
for (j=n; j >0 ; j = j / c ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}

```

Độ phức tạp hằng số $O(\text{Log}(\text{Log}(n)))$: nếu biểu thức khởi đầu lại của vòng lặp được nhân hoặc chia cho một hàm mũ.

Ví dụ 1.11. Đoạn code dưới đây có độ phức tạp $\text{Log Log}(n)$.

```

for (i=1; j<=n; j*= Pow(i, c) ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
for (j=n; j>=0; j = j- Function(j) ){ //Function(j) =sqrt(j) hoặc lớn hơn 2.
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}

```

Độ phức tạp của chương trình: độ phức tạp của một chương trình bằng số lần thực hiện một chỉ thị tích cực trong chương trình đó. Trong đó, một chỉ thị được gọi là tích cực trong chương trình nếu chỉ thị đó phụ thuộc vào độ dài dữ liệu và thực hiện không ít hơn bất kỳ một chỉ thị nào khác trong chương trình.

Ví dụ 1.12. Tìm độ phức tạp thuật toán sắp xếp kiểu Bubble-Sort?

```

Void Bubble-Sort ( int A[], int n ) {
    for ( i=1; i<n; i++) {
        for ( j = i+1; j<=n; j++){
            if (A[i] > A[j]){//đây chính là chỉ thị tích cực
                t = A[i]; A[i] = A[j]; A[j] = t;
            }
        }
    }
}

```

```

    }
  }
}

```

Lời giải. Sử dụng trực tiếp nguyên lý cộng ta có:

- Với $i = 1$ ta cần sử dụng $n-1$ phép so sánh $A[i]$ với $A[j]$;
- Với $i = 2$ ta cần sử dụng $n-1$ phép so sánh $A[i]$ với $A[j]$;
-
- Với $i = n-1$ ta cần sử dụng 1 phép so sánh $A[i]$ với $A[j]$;

Vì vậy tổng số các phép toán cần thực hiện là:

$$S = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 \leq n^2 = O(n^2).$$

Ghi chú. Độ phức tạp thuật toán cũng là số lần thực hiện phép toán tích cực. Phép toán tích cực là phép toán thực hiện nhiều nhất đối với thuật toán.

3.3. Phương pháp sinh

Mô hình thuật toán sinh được dùng để giải lớp các bài toán liệt kê, bài toán đếm, bài toán tối ưu, bài toán tồn tại thỏa mãn hai điều kiện:

- **Điều kiện 1:** Có thể xác định được một thứ tự trên tập các cấu hình cần liệt kê của bài toán. Biết cấu hình đầu tiên, biết cấu hình cuối cùng.
- **Điều kiện 2:** Từ một cấu hình chưa phải cuối cùng, ta xây dựng được thuật toán sinh ra cấu hình đứng ngay sau nó.

Mô hình thuật toán sinh được biểu diễn thành hai bước: bước khởi tạo và bước lặp. Tại bước khởi tạo, cấu hình đầu tiên của bài toán sẽ được thiết lập. Điều này bao giờ cũng thực hiện được theo giả thiết của bài toán. Tại bước lặp, quá trình lặp được thực hiện khi gặp phải cấu hình cuối cùng. Điều kiện lặp của bài toán bao giờ cũng tồn tại theo giả thiết của bài toán. Hai chỉ thị cần thực hiện trong thân vòng lặp là đưa ra cấu hình hiện tại và sinh ra cấu hình kế tiếp. Mô hình sinh kế tiếp được thực hiện tùy thuộc vào mỗi bài toán cụ thể. Tổng quát, mô hình thuật toán sinh được thể hiện như dưới đây.

Thuật toán Generation;

begin

Bước1 (Khởi tạo):

<Thiết lập cấu hình đầu tiên>;

Bước 2 (Bước lặp):

while (<Lặp khi cấu hình chưa phải cuối cùng>) **do**

<Đưa ra cấu hình hiện tại>;

<Sinh ra cấu hình kế tiếp>;

endwhile;

End.

Ví dụ 1. Vector $X = (x_1, x_2, \dots, x_n)$, trong đó $x_i = 0, 1$ được gọi là một xâu nhị phân có độ dài n . Hãy liệt kê các xâu nhị phân có độ dài n . Ví dụ với $n=4$, ta sẽ liệt kê được 24 xâu nhị phân độ dài 4 như trong Bảng 2.1.

Bảng 2.1. Các xâu nhị phân độ dài 4

STT	$X=(x_1, x_2, x_3, x_4)$	STT	$X=(x_1, x_2, x_3, x_4)$
0	0 0 0 0	8	1 0 0 0
1	0 0 0 1	9	1 0 0 1
2	0 0 1 0	10	1 0 1 0
3	0 0 1 1	11	1 0 1 1
4	0 1 0 0	12	1 1 0 0
5	0 1 0 1	13	1 1 0 1
6	0 1 1 0	14	1 1 1 0
7	0 1 1 1	15	1 1 1 1

Lời giải:

Điều kiện 1: Gọi thứ tự của xâu nhị phân $X=(x_1, x_2, \dots, x_n)$ là $f(X)$. Trong đó, $f(X)=k$ là số chuyển đổi xâu nhị X thành số ở hệ cơ số 10. Ví dụ, xâu $X = (1, 0, 1, 1)$ được chuyển thành số hệ cơ số 10 là 11 thì ta nói xâu X có thứ tự 11. Với cách quan niệm này, xâu đứng sau xâu có thứ tự 11 là 12 chính là xâu đứng ngay sau xâu $X = (1, 0, 1, 1)$. Xâu đầu tiên có thứ tự là 0 ứng với xâu có n số 0. Xâu cuối cùng có thứ tự là 2^n-1 ứng với xâu có n số 1. Như vậy, điều kiện 1 của thuật toán sinh đã được thỏa mãn.

Điều kiện 2: Về nguyên tắc ta có thể lấy $k = f(X)$ là thứ tự của một xâu bất kỳ theo nguyên tắc ở trên, sau đó lấy thứ tự của xâu kế tiếp là $(k + 1)$ và chuyển đổi $(k+1)$ thành số ở hệ cơ số 10 ta sẽ được xâu nhị phân tiếp theo. Xâu cuối cùng sẽ là xâu có n số 1 ứng với thứ tự $k = 2^n-1$. Với cách làm này, ta có thể coi mỗi xâu nhị phân là một số, mỗi thành phần của xâu là một bit và chỉ cần cài đặt thuật toán chuyển đổi cơ số ở hệ 10 thành số ở hệ nhị phân. Ta có thể xây dựng thuật toán tổng quát hơn bằng cách xem mỗi xâu

nhị phân là một mảng các phần tử có giá trị 0 hoặc 1. Sau đó, duyệt từ vị trí bên phải nhất của xâu nếu gặp số 1 ta chuyển thành 0 và gặp số 0 đầu tiên ta chuyển thành 1. Ví dụ với xâu $X = (0, 1, 1, 1)$ được chuyển thành xâu $X = (1, 0, 0, 0)$, xâu $X = (1, 0, 0, 0)$ được chuyển thành xâu $X = (1, 0, 0, 1)$. Lời giải và thuật toán sinh xâu nhị phân kế tiếp được thể hiện trong chương trình dưới đây. Trong đó, thuật toán sinh xâu nhị phân kế tiếp từ một xâu nhị phân bất kỳ là hàm *Next_Bits_String()*.

```
#include <iostream>
#include <iomanip>
#define MAX 100
using namespace std;
int X[MAX], n, dem = 0; //sử dụng các biến toàn cục X[], n, OK, dem
bool OK = true;
void Init(void){ //khởi tạo xâu nhị phân đầu tiên
    cout<<"Nhập n="; cin>>n;
    for(int i = 1; i<=n; i++) //thiết lập xâu với n số 0
        X[i]=0;
}
void Result(void){ //đưa ra xâu nhị phân hiện tại
    cout<<"\n Xâu thứ "<<++dem<<":";
    for(int i=1; i<=n; i++)
        cout<<X[i]<<setw(3);
}
void Next_Bits_String(void){ //thuật toán sinh xâu nhị phân kế tiếp
    int i=n;
    while(i>0 && X[i]){ //duyet từ vị trí bên phải nhất
        X[i]=0; //nếu gặp X[i] = 1 ta chuyển thành 0
        i--; //lùi lại vị trí sau
    }
    if (i>0) X[i]=1; //gặp X[i] = 0 đầu tiên ta chuyển thành 1
    else OK = false; //kết thúc khi gặp xâu có n số 1
}
int main(void){ //đây là thuật toán sinh
    Init(); //thiết lập cấu hình đầu tiên
    while(OK){ //lặp khi chưa phải cấu hình cuối cùng
        Result(); //đưa ra cấu hình hiện tại
        Next_Bits_String(); //sinh ra cấu hình kế tiếp
    }
}
```

}

Ví dụ 2. Liệt kê tập con m phần tử của tập n phần tử. Cho $X = \{ 1, 2, \dots, n \}$. Hãy liệt kê tất cả các tập con k phần tử của X ($k \leq n$).

Lời giải: Mỗi tập con của tập hợp X có thể biểu diễn bằng bộ có thứ tự gồm k thành phần $a = (a_1 a_2 \dots a_k)$ thỏa mãn $1 \leq a_1 \leq a_2 \leq \dots \leq a_k \leq n$. Trên tập các tập con k phần tử của X có thể xác định nhiều thứ tự khác nhau. Thứ tự dễ nhìn thấy nhất là thứ tự từ điển được định nghĩa như sau:

Ta nói tập con $a = a_1 a_2 \dots a_k$ đi trước tập con $a' = a'_1 a'_2 \dots a'_k$ trong thứ tự từ điển và ký hiệu là $a < a'$, nếu tìm được chỉ số j ($1 \leq j \leq k$) sao cho

$$a_1 = a'_1, a_2 = a'_2, \dots, a_{j-1} = a'_{j-1}, a_j < a'_j.$$

Chẳng hạn $X = \{ 1, 2, 3, 4, 5 \}$, $k = 3$. Các tập con 3 phần tử của X được liệt kê theo thứ tự từ điển như sau:

1	2	3
1	2	4
1	2	5
1	3	4
1	3	5
1	4	5
2	3	4
2	3	5
2	4	5
3	4	5

Như vậy, tập con đầu tiên trong thứ tự từ điển là $(1, 2, \dots, k)$ và tập con cuối cùng là $(n-k+1, n-k+2, \dots, n)$. Giả sử $a = (a_1, a_2, \dots, a_k)$ là tập con hiện tại và chưa phải là cuối cùng, khi đó có thể chứng minh được rằng tập con kế tiếp trong thứ tự từ điển có thể được xây dựng bằng cách thực hiện các qui tắc biến đổi sau đối với tập con đang có.

- ☐ Tìm từ bên phải dãy $a_1, a_2, \dots, a_k$ phần tử $a_i \neq n - k + i$
- ☐ Thay a_i bởi $a_i + 1$,
- ☐ Thay a_j bởi $a_i + j - i$, với $j := i+1, i+2, \dots, k$

Chẳng hạn với $n = 6, k = 4$. Giả sử ta đang có tập con $(1, 2, 5, 6)$, cần xây dựng tập con kế tiếp nó trong thứ tự từ điển. Duyệt từ bên phải ta nhận được $i = 2$, thay a_2 bởi $a_2 + 1 = 2 + 1 = 3$. Duyệt j từ $i + 1 = 3$ cho đến k , ta thay thế $a_3 = a_2 + 3 - 2 = 3 + 3 - 2 = 4$, $a_4 = a_2 + 4 - 2 = 3 + 4 - 2 = 5$ ta nhận được tập con kế tiếp là $(1, 3, 4, 5)$.

Với qui tắc sinh như trên, chúng ta có thể mô tả bằng thuật toán sau:

Thuật toán liệt kê tập con kế tiếp m phần tử của tập n phần tử:

```
void Next_Combination(void){
```